Artificial Life with Robotics
01 - Introduction to NetLogo

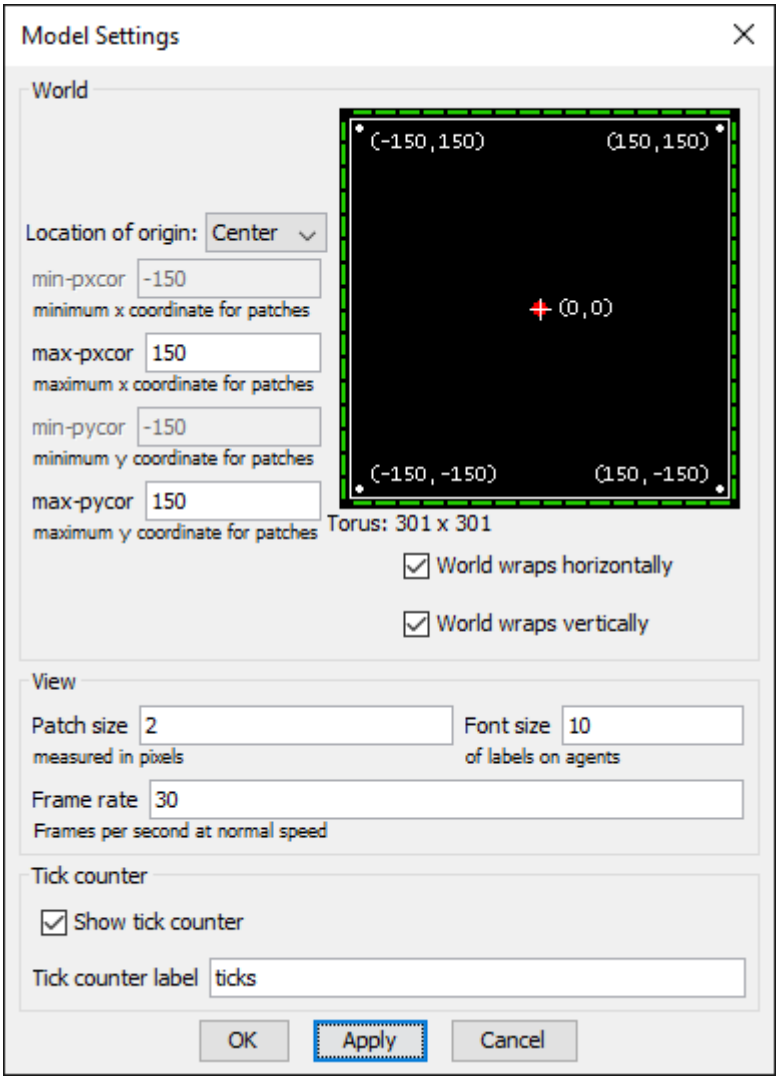
Tutorial Contents

Step 1 - Setting up the environment 3
Step 2 – Creating the inhabitants 4
Step 3 – creating agent variables and global variables using code 4
Step 4 – creating an interface and global variables 4
Step 4.1 – Creating a global variable with a slider 4
Step 4.2 – Creating a global variable with a switch 4
Step 4.3 – Creating command buttons 5
Step 5 – creating the setup function 6
Step 6 – creating the go function 7
Step 7 – making the people move 7
Step 8 – test your model and experiment with the parameters 7
Step 9 – setting up the butterflies 7
Step 9.1 – making the butterflies move and setting up local agent variables 7
Step 9.2 – setup make the butterfly see by setting up a vision cone 7
Step 9.3 – resetting the patch colors to black 8
Step 9.4 – creating a collision detection radius to detect collisions with people 8
Step 9.5 – updating the hit counters and making the butterflies reactive 8
Step 10 – Observe and play 9
Step 11 – adding another type of agent (food) and more variables 10
Step 12 – making our butterflies unique 10
Step 12.1 – creating a vison cone function 10
Step 12.2 – updating the names of the vision cones in the rest of the code 10
Step 12.3 – updating the speed section of the code for the butterflies 10
Step 13 – adding food to our model by calling a function 11
Step 14 – continually growing food in the model 11
Step 15 – reorganising our code to make it easier to manage 11
Step 15.1 – moving the visualisation aspects to a visualisation function 11
Step 15.2 – moving the people aspects to a separate function 12
Step 16 – creating a food function for the butterflies 12
Step 17 – switching your butterflies behaviours 13
Step 18 – creating venom pods 13
Step 19 – picking up venom pods 13
Step 19.1 – adding another variable to the butterflies 13
Step 19.2 – creating the pickup function 14
Step 19.3 – making the vpods have impact 14
Step 20 – using graphs to keep tabs on our model 15

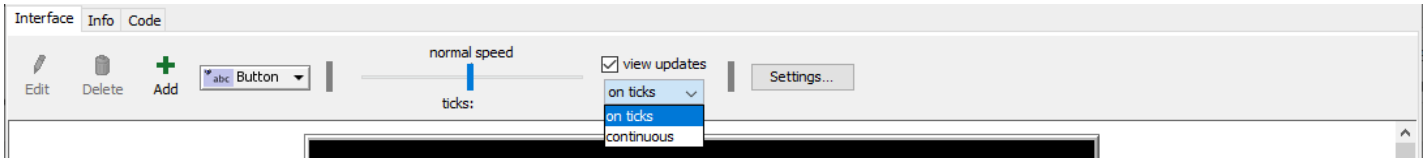
Step 21 – fixing an error in our existing code 15
Step 22 – creating more global variable to make our model more flexible 16
Step 22.1 – setting our variables up 16
Step 23 – adjusting the timer and adding functionality 16
Step 24 – creating day and night in our world 16
Step 25 – making day and night have an impact on the butterflies vision 16
Step 26 – creating buildings in the environment 17
Step 26.1 – creating a function to setup our buildings 17
Step 26.2 – fixing the color changing issue 17
Step 26.3 – making the object solid to our agents 18

Step 1 - Setting up the environment

- Open NetLogo and save the file in an appropriate location (student drive)
- Click on the setting button and modify the environment to:
    a. extend 150 patches from the centre both horizontally and vertically
    b. wraps both vertically and horizontally by ensuring boxes are ticked (creating infinate world)
    c. set the patch size to 2
    d. set framerate to 30
    e. show the tick counter



Once this is done on the interface set the make sure view updates is ticked and ensure the world updates on ticks rather than continuously as shown below.



Step 2 – Creating the inhabitants

In this world there will be 2 types of inhabitant's people and butterflies. To create these types of populations, enter the following code in the code tab:

```
breed [ peoples people ]
breed [ butterflies butterfly ]
```

Step 3 – creating agent variables and global variables using code

In this model we want the butterflies to keep a log of the number of people they have avoided and the number of people they have hit. In addition we will create a global variable called rad to demonstrate how these work. To create these variables use the following code:

```
butterflies-own [ people_seen people_hit ]

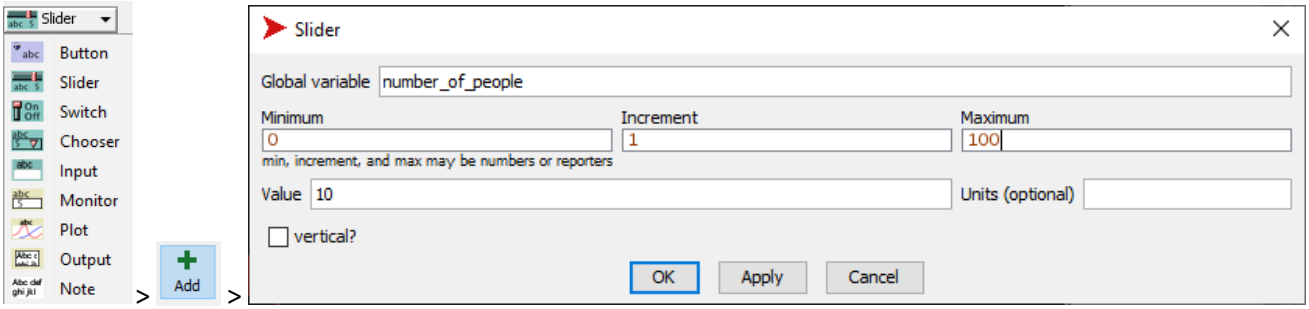
globals [rad]
```

Step 4 – creating an interface and global variables

In this model we want to be able to easily change the parameters of the model, we will therefore create some global variables and buttons to activate commands easily in the interface tab. Create the

Step 4.1 – Creating a global variable with a slider

Select slider from the option list then click the add button and the slider dialogue box will appear where you can name the variable and enter the constraints and default value of the variable.

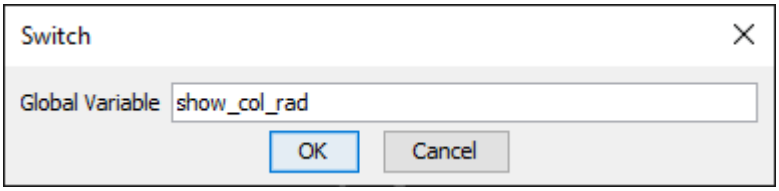


Create the following sliders with the stated parameters:

Name	Min	Incr	Max	Val
number_of_people	1	1	100	40
people_speed	1	1	10	1
pwr	10	1	180	10
number_of_butterflies	1	1	20	5
butterflies_speed	1	1	10	1
bwr	10	1	180	10
vis_rad	0	1	50	25
vis_ang	0	1	180	45

Step 4.2 – Creating a global variable with a switch

Select switch from the option list then click the add button and the switch dialogue box will appear where you can name the variable.

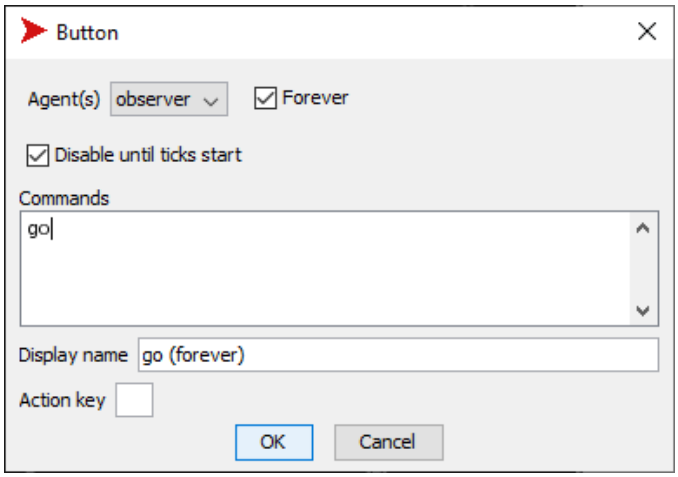


Create the following switches with the stated parameters:

- show_col_rad
- show_vis_cone

Step 4.3 – Creating command buttons

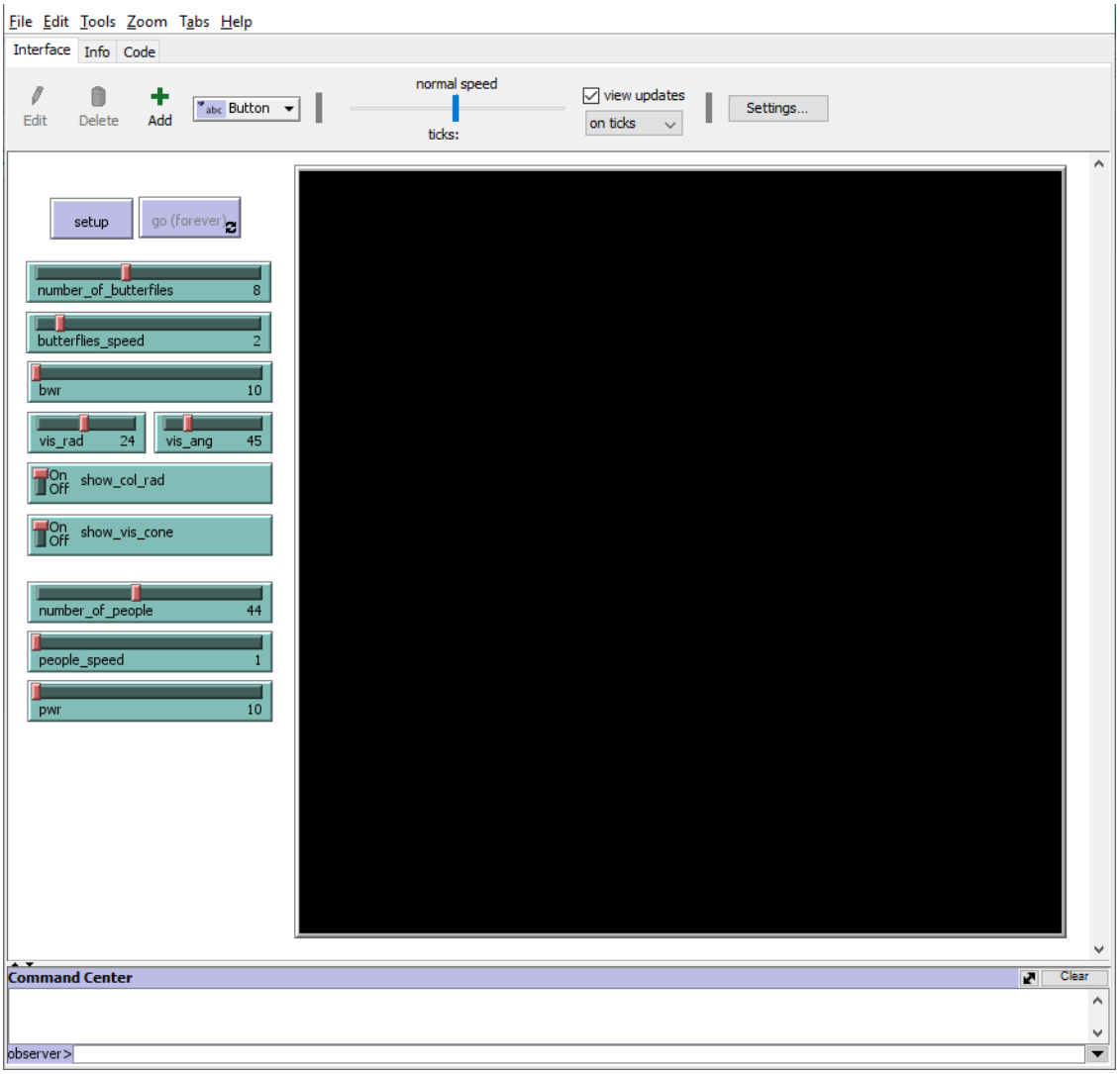
Select button from the option list then click the add button and the button dialogue box will appear where you can name the variable and sent the command for it to activate.



Create the following buttons with the stated parameters:

Commands	Agent(s)	Forever	Disable until ticks start
go	Observer	Ticked	Ticked
setup	Observer	Unticked	

Once these steps have been completed arrange the interface as follows:



Step 5 – creating the setup function

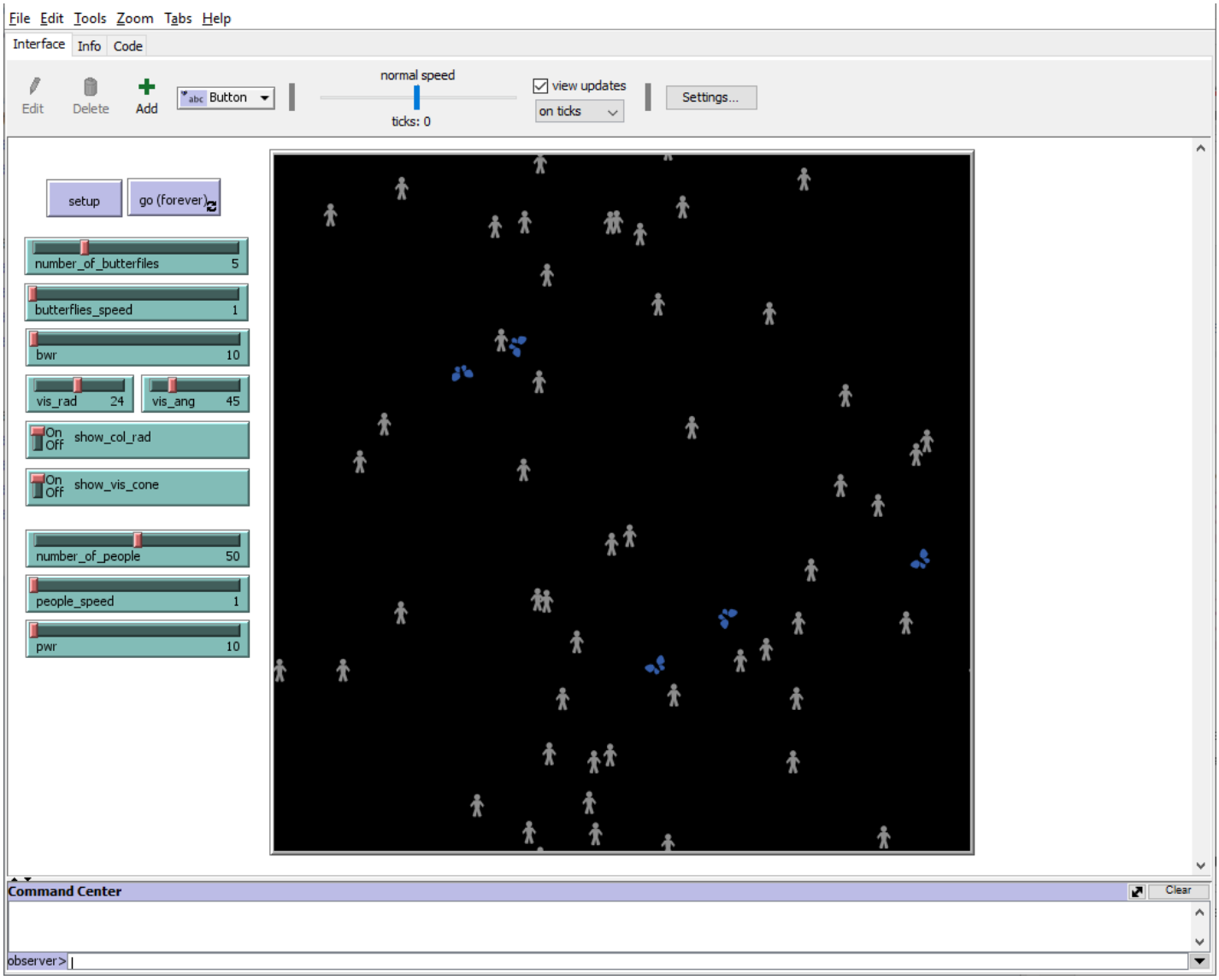
The following code is used to setup the world which will reset it, place the inhabitants within the world and configure them.

```
to setup                                ; this creates a function called setup
  clear-all                             ; this clears the world of any previous activities
  reset-ticks                           ; this resets the ticks counter
  set rad 5                             ; this sets the global variable rad to 3

  create-peoples number_of_people [     ; this creates the number of people that your global variable states
    setxy random-xcor random-ycor       ; this sets the starting position of the people to a random location in the world
    set color gray                      ; this sets the color of the people to gray
    set size 10                         ; this sets the size of the people to 10
    set shape "person"                  ; this sets the shape of the people to a person
  ]

  create-butterflies number_of_butterfiles [; this creates the number of butterflies that your global variable states
    setxy random-xcor random-ycor       ; this sets the starting position of the butterflies to a random location in the world
    set color blue                      ; this sets the color of the butterflies to blue
    set size 10                         ; this sets the size of the butterflies to 10
    set shape "butterfly"               ; this sets the shape of the butterflies to a butterfly
  ]
end
```

If you have entered this correctly when you go back to the interface and click the setup button something similar the this should appear:



Step 6 – creating the go function

In this example we will use several different functions in order to demonstrate the principle. Type the following code but comment out reset_patch_color and make_butterflies_move as we will test the system incrementally.

```
to go                                ; this creates a function called go
  make_people_move                   ; this calls the make_people_move function
  reset_patch_color                  ; this calls the reset_patch_color function
  make_butterflies_move              ; this calls the make_butterflies_move function
  tick                               ; this adds 1 to the tick counter
end
```

Step 7 – making the people move

Creating this function will make the people agents within your environment move. The comments within the code explain exactly what each line does.

```
to make_people_move                  ; this creates a function called make_people_move
  ask peoples [                      ; this asks all of the people in the population to do what is in the brackets
    set color gray                   ; this sets the color of each person to gray
    right ( random pwr - ( pwr / 2))  ; this turns the person right relative to its current heading by a random degree number
    forward people_speed             ; this sets the speed at which the people move
  ]
end
```

Step 8 – test your model and experiment with the parameters

Test your model after entering the code and you will see that the agents will move around using the parameters set within the sliders. To add more agents to the model you will need to press the setup button again after moving the slider.

Step 9 – setting up the butterflies

Step 9.1 – making the butterflies move and setting up local agent variables

The following code will setup some local variables which we will use later and make the butterflies move forward in a single direction. NOTE: uncomment the make_butterflies_move function in the go function.

```
to make_butterflies_move             ; this is defining a function called make_butterflies_move
  ask butterflies [                  ; this asks all of the butterflies in the population to do what is in the brackets

    set color blue                   ; this sets the color of each butterfly to blue
    let seen [false]                 ; this creates a local variable called seen
    let hit [false]                  ; this creates a local variable called hit

    forward butterflies_speed        ; moves butterfly forward by the butterflies_speed variable
  ]
end
```

Test the code in the interface and you will see the butterflies just go straight. The next step is to make the butterfly see.

Step 9.2 – setup make the butterfly see by setting up a vision cone

The top part of the code sets up a field of view in front of each butterfly. The lower section visualises the cone by changing the color of the patches. Add this code under the local variables within the make_butterflies_move function

```
    ask peoples in-cone vis_rad vis_ang [   ; this sets up a vison cone with the parameters from vis_rad vis_ang to detects people
      set color green                       ; this sets the color of the person detected within the vision code of the butterfly to green
      set seen true                         ; this sets the local variable called seen to true indicating that a person has been seen
    ]

    if show_vis_cone = true [               ; this will switch on the visualisation of the vision cone if the switch is set to true
      ask patches in-cone vis_rad vis_ang [ ; this sets up a vision cone to display the size of the cone by changing the patch color
        set pcolor red                      ; this sets the patch color to red
      ]
    ]
```

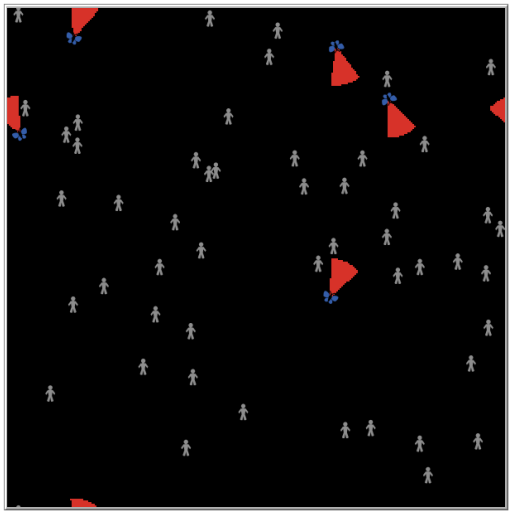
Once you have entered this code see what happens when you run it in the interface. You will notice that the vision cone leaves a trail which is not particularly helpful. To address this issue you will need to add some code to return the patch colors to black.

Step 9.3 – resetting the patch colors to black

In the go function uncomment the reset_patch_color function then add the following function below the go function (ensure it is outside of the go function!)

```
to reset_patch_color
  ask patches [                          ; this asks all of the patches in the population to do what is in the brackets
    set pcolor black                     ; this sets the color of each patch to black
  ]
end
```

Run the model again and see what happens, the vison cone should now work correctly.



Step 9.4 – creating a collision detection radius to detect collisions with people

In order to detect collisions with people we will setup a circle around the butterflies with the following code just below the vison cone in the make_butterflies_move function. The upper part detects collisions with people the lower part creates a visualisation.

```
    ask peoples in-radius rad [          ; this sets up a radius around the butterfly for collision detection with people using rad
      set hit true                       ; this sets the local variable called hit to true indicating that a person has collided with
    ]

    if show_col_rad = true [             ; this will switch on the visualisation of the collision radius if the switch is set to true
      ask patches in-radius rad [        ; this sets up a radius around the butterfly to display the size of the collison radius
        set pcolor orange                ; this sets the patch color to orange
      ]
    ]
```

Once you have entered this code run the model in the interface to see what happens.

Step 9.5 – updating the hit counters and making the butterflies reactive

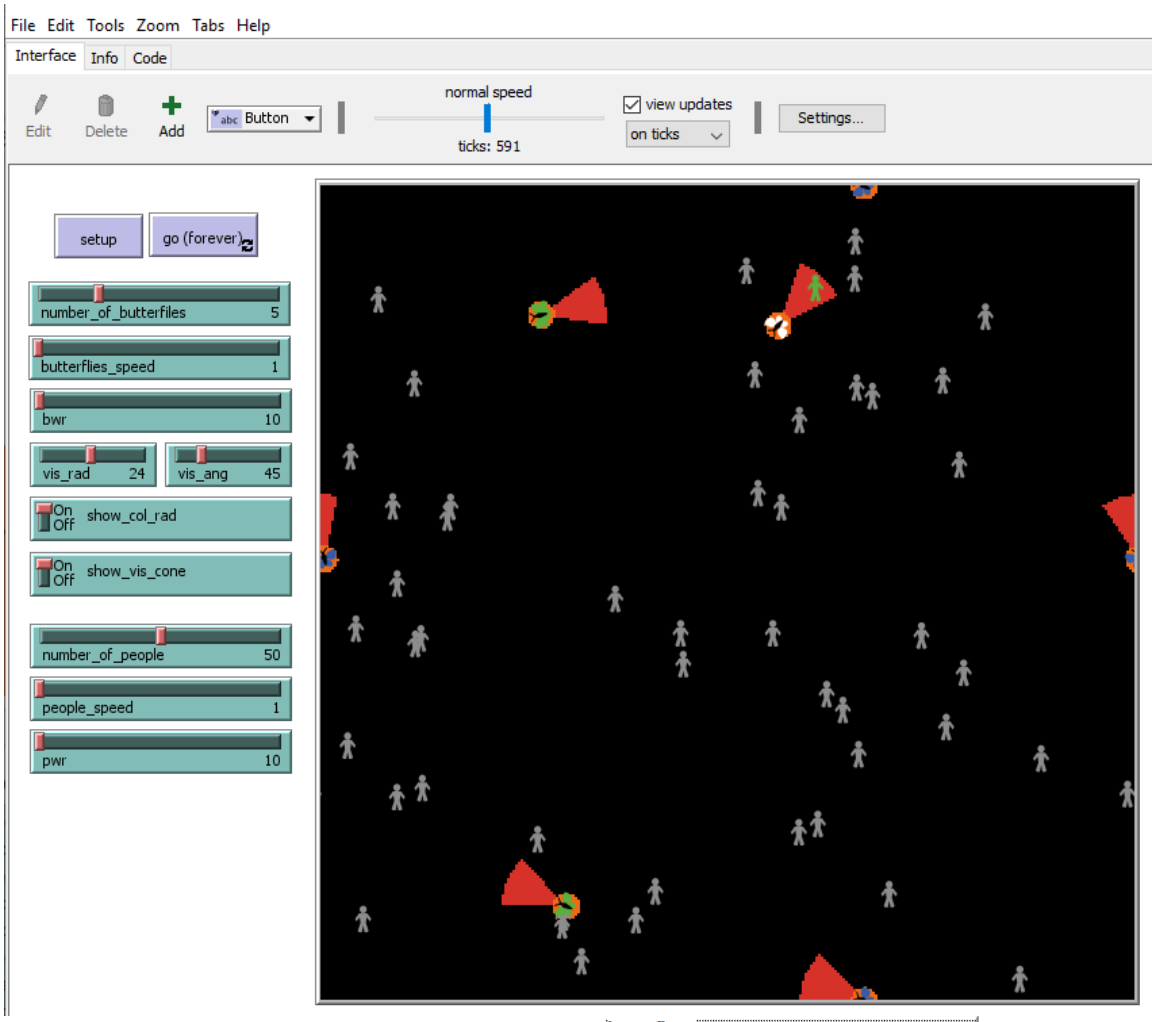
The following code uses if statements based on the local variables to update the behaviour and hit counters of the butterflies. Add this code below the collision detection code.

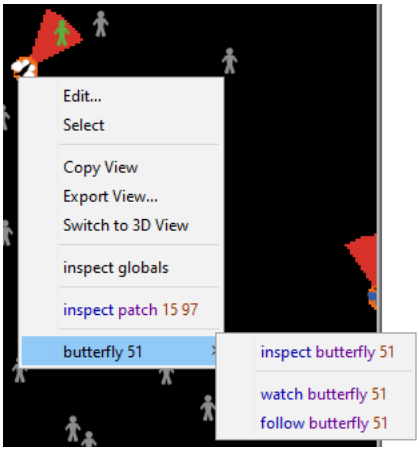
```
    ifelse seen = true [                 ; if then else statement based on the local variable seen, if seen = true then...
      set people_seen people_seen + 1    ; add 1 to the people_seen count
      set color white                    ; set color of butterfly to white
      right 180                          ; set heading of the butterfly to 180 (turn around to avoid!)
    ][                                   ; if seen = false...
      right (random bwr - (bwr / 2))     ; this turns the butterfly right relative to its current heading by a random degree number
    ]

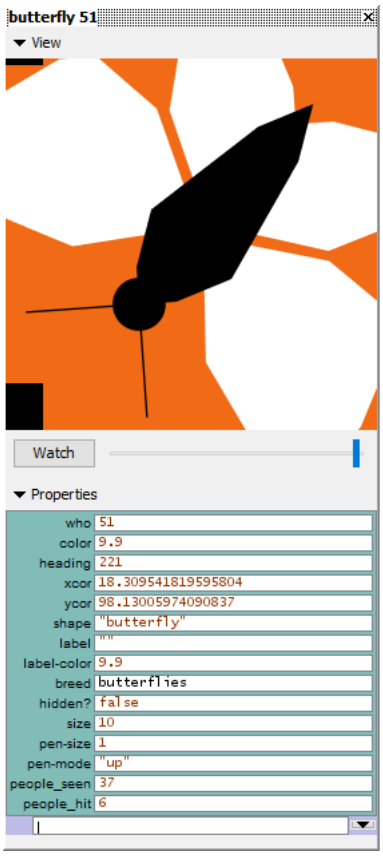
    if hit = true [                      ; if statement based on the local variable hit, if seen = true then...
      set people_hit people_hit + 1      ; add 1 to the people_hit count
      set color green                    ; et color of butterfly to green
    ]
```

Step 10 – Observe and play

Once you have added this code run the model in the interface. The butterflies should try to avoid the people when they see them. By stopping the model and right clicking on a butterfly you can see the variables of that butterfly. You can also click watch to make it easier to track the butterfly







Step 11 – adding another type of agent (food) and more variables

Building on the existing code enter the additional lines to add food agents and more variables to the butterflies.

```
breed [ peoples people ]                    ; creating a population of people who will move around aimlessly
breed [ butterflies butterfly ]             ; creating a population of butterflies who will move around aimlessly but also seen the people
breed [ food flowers ]

butterflies-own [ people_seen people_hit    ; this creates 2 variables which will be used to count the total people seen and total people hit
health robustness speed_variation           ; this creates 3 variables for health, durability and speed
per_vis_rad per_vis_ang                     ; this creates variables for personalised vision cones
food_around_me closest_food                 ; this creates 2 variables to save the locations of food
]

food-own [ amount ]                         ; this creates a variable for the food to establish amount of the resource

globals [rad]                               ; this creates a global variable called rad
```

Step 12 – making our butterflies unique

To make our butterflies unique add the following lines to the create-butterflies section of code in the setup.

```
  create-butterflies number_of_butterfiles [; this creates the number of butterflies that your global variable states determined by the slider
    setxy random-xcor random-ycor           ; this sets the starting position of the butterflies to a random location in the world
    set color blue                          ; this sets the color of the butterflies to blue
    set size 10                             ; this sets the size of the butterflies to 10
    set shape "butterfly"                   ; this sets the shape of the butterflies to a butterfly

    ; making our butterflies unique
    set health 50 + random 50               ; this sets the health of the butterfly by adding 50 + a random alloctaion up to 50
    adjust_vision_cone                      ; this calls the adjust_vision_cone fuction to setup the vision cone
    set robustness random 10                ; this sets the robustness variable to a random value up to 10. lower means the butterfly is less a
    set speed_variation random 10           ; this sets the speed_variation variable to a random value up to 10. the higher the value the faste
    set heading 0                           ; this sets the starting heading of the butterfly to 0 (for demonstartion of speed difference)
    pen-down                                ; this puts the pen down so you can see where the butterfly moves
  ]
```

Step 12.1 – creating a vison cone function

After this create a function called adjust_vision_cone to customise the vision cone of each butterfly. The plan is to create a vision cone that is slightly random but also considers the health of the butterfly (I.e. if its not as healthy it cannot see as far or as wide). To do this enter the following code.

```
to adjust_vision_cone                                   ; this creates a function called adjust_vision_cone
  if ((vis_rad + random 20)*(health * 0.01)) > 0 [      ; if the calculation if greater than 0 then...
    set per_vis_rad ((vis_rad + random 20)*(health * 0.01))  ; set the personal vision radius to factor in some randomness and health
  ]
  if ((vis_ang + random 20)*(health * 0.01)) > 0 [      ; if the calculation if greater than 0 then...
    set per_vis_ang ((vis_ang + random 20)*(health * 0.01))  ; set the personal vision angle to factor in some randomness and health
  ]
end
```

Step 12.2 – updating the names of the vision cones in the rest of the code

Where there was previously a standard vision cone we now have a unique vision cone for each butterfly. The final part of implementing this new vision cone is to change the vis_rad and vis_ang to per_vis_rad and per_vis_ang wherever it has been used outside of the vision cone function. You could find these using CTRL F or look manually. They will be located where you are looking for people and in the visualisation sections of your code.

Step 12.3 – updating the speed section of the code for the butterflies

In the make_butterflies_move function modify the forward butterflies_speed code to the following:

```
    forward butterflies_speed + ( speed_variation * 0.1 ) ; moves butterfly forward by the butterflies_speed variable
```

* Now setup and run the model to see what happens

Step 13 – adding food to our model by calling a function

To add food agents to our world we will create a function that we can later use to continually add food to the environment. Write the following function below (not inside of!) the setup function.

```
to grow_food
  setxy random-xcor random-ycor             ; this sets the position of the food to a random location in the world
  set color yellow                          ; this sets the color of the food to yellow
  set size 10                               ; this sets the size of the food to 10
  set shape "plant"                         ; this sets the shape of the food to a plant
  set amount random 100                     ; this sets the amount of food per plant to a random value up to 100
end
```

Then add the following lines within the setup function.

```
  create-food 10 [                          ; this creates X number of new food plants for the butterflies to feed from
    grow_food                               ; this calls the grow_food function
  ]
```

* Now run the model and see what happens. You should have plants (food sources appear in your world).

Step 14 – continually growing food in the model

To continually add food to our model we can create a function to sprout new food with the following code.

```
to grow_more_food                       ; this creates a function called grow_more_food
  if ticks > 1000 [                     ; if the current number of ticks is greater than 100 then...
    ask patch random-xcor random-ycor [ ; ask a (1) patch in a random location (x, y coordinate) to do the following...
      sprout-food 1 [grow_food]         ; sprout (create new) food (1 in this instance) then call the grow_food function to set the parameters
    ]
    reset-ticks                         ; this resets the ticks counter back to 0
  ]
end
```

In order to enable this function add the following line to the go function.

```
  grow_more_food                        ; this calls the grow_more_food function
```

* now test the model and see what happens. Food should start growing in your world every 1000 ticks.

Step 15 – reorganising our code to make it easier to manage

Since our code is going to get a lot longer we want to make it easy to manage and to read. In order to do this we will break some of the existing code up into sub-functions as follows.

Step 15.1 – moving the visualisation aspects to a visualisation function

Cut and paste the visualisation code from the make_butterflies_move function into a new function called show_visualisations. The function should look as follows:

```
to show_visualisations                          ; this creates a function called show_visualisations
  if show_col_rad = true [                      ; this will switch on the visualisation of the collision radius if the switch is set to true
    ask patches in-radius rad [                 ; this sets up a radius around the butterfly to the value of the global variable rad which we
      set pcolor orange                         ; this sets the patch color to orange
    ]
  ]
  if show_vis_cone = true [                     ; this will switch on the visualisation of the vision cone if the switch is set to true
    ask patches in-cone per_vis_rad per_vis_ang [ ; this sets up a vision cone in front of the butterfly to the value of the global variables pe
      set pcolor red                            ; this sets the patch color to red
    ]
  ]
end
```

Now in place of this code in the make_butterflies_move function inside of the ask butterflies code to call the new function you have just created.

```
    show_visualisations                         ; call the show_visualisations function
```

* now test the model to check it still works

Step 15.2 – moving the people aspects to a separate function

Cut and paste the people aspects of the code into a new function called people function as per the instructions below. NOTE: this will be a return function so you will need to write **to-report** at the beginning of the function. The code should look SIMILAR to the following:

```
to-report people_function                          ; this creates a reporting function called people_function
  let seen [false]                                 ; this creates a local variable called seen
  let hit [false]                                  ; this creates a local variable called hit
  ask peoples in-cone per_vis_rad per_vis_ang [    ; this sets up a vison cone on the butterfly with the parameters from per_vis_rad
    set color green                                ; this sets the color of the person detected within the vision code of the butterf
    set seen true                                  ; this sets the local variable called seen to true indicating that a person has be
  ]

  ask peoples in-radius rad [                      ; this sets up a radius around the butterfly to the value of the global variable r
    set hit true                                   ; this sets the local variable called hit to true indicating that a person has col
  ]

  ifelse seen = true [                             ; if then else statement based on the local variable seen, if seen = true then...
    set people_seen people_seen + 1                ; add 1 to the people_seen count
    set color white                                ; set color of butterfly to white
    right 180     ;--------------------------------; set heading of the butterfly to 180 (turn around to avoid!)
  ][                                               ; if seen = false...
    right (random bwr - (bwr / 2)) ;---------------; this turns the butterfly right relative to its current heading by a random degre
  ]

  if hit = true [                                  ; if statement based on the local variable hit, if seen = true then...
    set people_hit people_hit + 1                  ; add 1 to the people_hit count
    set color green                                ; set color of butterfly to green
    set health health - robustness ;+++++++++++++++; adjust health of butterfly to health - collision penulty (robustness)
    adjust_vision_cone ;++++++++++++++++++++++++++; calls adjust_vision_cone to update the vision parameters based on heath changes
  ]
  report seen ;++++++++++++++++++++++++++++++++++; return true or false based in local variable seen
end
```

NOTE: you will eventually comment out the --- bits of code but leave in place for now. You should add the +++ bits of code as we will use these later.

Because this new function returns a binary value we need to create a local variable and call the newly created function to fill that variable. Enter the following line in place of the original code.

```
    let have_seen_person people_function           ; this creates a local variable called have_seen_person the fills it with the return
```

* now test the model to check it still works

Step 16 – creating a food function for the butterflies

We will now create a function to enable our butterflies to smell food within a particular range and return a value to a local variable within the make_butterfly_move function in order to decide what the butterfly should do. Add the following code to your program.

```
to-report food_function [sensitivity]                          ; this creates a reporting function called food_function and expects a value for ser
  set food_around_me other ( food in-radius sensitivity )      ; this sets the food_around_me variable to the ID's of the food within the sensitivi
  set closest_food min-one-of food_around_me [distance myself] ; this sets the closest_food variable to the ID of the closest food source
  let can_smell_food [false]                                   ; this creates a local variable called can_smell_food and sets it to false
  let eating_food [false]                                      ; this creates a local variable called eating_food and sets it to false

  if health < 100 [                                            ; if health is less than 100 then...
    ask food in-radius rad [                                   ; this sets up a radius around the food to the value of the global variable rad whic
      ifelse amount > 0 [                                      ; if amount (a food variable) is greater than 0...
        set eating_food true                                   ; set the local variable called eating_food to true indicating the butterfly is eati
        set amount amount - 5                                  ; reduces 5 from the amount variable in the food
        set color color - .25                                  ; reduce the color intensity of the food by .25
      ][                                                       ; otherwise...
        die                                                    ; there is no food left so kill the good agent
      ]
    ]
  ]

  if eating_food = true [                                      ; if eating_food is true then...
    set health health + 5                                      ; add 5 to health of butterfly
    adjust_vision_cone                                         ; call adjust_vision_cone function as health impact on the vison of the butterfly
  ]

  if (closest_food != nobody) [                                ; if closest_food is not empty (the butterfly can smell food in range) then...
    set can_smell_food true                                    ; set can_smell_food to true
  ]
  report can_smell_food                                        ; return value of can_smell_food to location where function was called
end
```

Add the following line to the code in the make_butterfly_move code. The 30 signifies the sense of smell.

```
    let can_smell_food food_function 30            ; this creates a local variable called can_smell_food then fills it with the return c
```

Step 17 – switching your butterflies behaviours

To switch the behaviour of your butterflies between avoiding people, looking for food and random wandering the code should look as follows:

```
to make_butterflies_move                                    ; this is defining a function called make_butterflies_move
  ask butterflies [                                         ; this asks all of the butterflies in the population to do what is in the brackets
    ifelse health > 0 [                                     ; if health is greater than 0 then (still alive)...
      show_visualisations                                   ; call the show_visualisations function
      set color blue                                        ; this sets the color of each butterfly to blue
      let have_seen_person people_function                  ; this creates a local variable called have_seen_person the fills it with the return
      let can_smell_food food_function 30                   ; this creates a local variable called can_smell_food then fills it with the return o
      ifelse ( have_seen_person = true ) [                  ; if local variable have_seen_person is true...
        right 180                                           ; set heading of the butterfly to 180 (turn around to avoid!)
        ][                                                  ; otherwise...
        ifelse ( can_smell_food = true ) and ( health < 100) [ ; if local variable can_smell_food is true...
          set heading ( towards closest_food )              ; set heading towards closest food source
          ][                                                ; otherwise...
          right (random bwr - (bwr / 2))                    ; this turns the butterfly right relative to its current heading by a random degree n
      ]]
      forward butterflies_speed + ( speed_variation * 0.1 ) ; moves butterfly forward by the butterflies_speed variable
    ][                                                      ; otherwise...
      set color gray                                        ; set color to gray to indicate dead butterfly
      die                                                   ; this kills off the butterfly
    ]
  ]
end
```

In addition to this you will need to comment out 2 lines in the people_function as outlined below.

```
    ;right 180     ;------------------------------; set heading of the butterfly to 180 (turn around to avoid!)
  ][                                               ; if seen = false...
    ;right (random bwr - (bwr / 2)) ;--------------; this turns the butterfly right relative to its current heading by a random degree number using
  ]
```

* Now test your model and experiment with the various parameters

Step 18 – creating venom pods

Similar to previous steps to create weapons we will breed a new population of agents called vpods as follows.

```
breed [ venom vpod ]                    ; creating a population of venom pods as weapons to defend the butterflies
```

The we will create a function to setup the vpods using the following code.

```
to make_venom                           ; this creates a function called make_venom
  setxy random-xcor random-ycor         ; this sets the position of the venom to a random location in the world
  set color green                       ; this sets the color of the venom to green
  set size 5                            ; this sets the size of the venom to 5
  set shape "x"                         ; this sets the shape of the venom to an x
end
```

We then add some lines to the setup function to add the vpods to the world and set them up by calling the make_venom function.

```
  create-venom 20 [                     ; this creates x number of new venom pods for the butterflies to store and use
    make_venom                          ; this calls the make_venom function
  ]
```

* Now test your model and see if the vpods appear

Step 19 – picking up venom pods

Step 19.1 – adding another variable to the butterflies

In order for the butterflies to keep a log on the vpods they are carrying you need to create an additional variable on the butterfly at the beginning of the code. The butterflies-own variables should now look like this.

```
butterflies-own [ people_seen people_hit    ; this creates 2 variables which will be used to count the total people seen and total people hit by
health robustness speed_variation           ; this creates 3 variables for health, durability and speed
per_vis_rad per_vis_ang                     ; this creates variables for personalised vision cones
food_around_me closest_food                 ; this creates 2 variables to save the locations of food
have_venom                                  ; this creates a variable to store the amount of venom held
]
```

Step 19.2 – creating the pickup function

To enable the butterflies to pickup the vpods we will write a new function similar to what we created for the food function but simpler as we are not actively looking. The code is as follows.

```
to pickup_venom                                 ; this creates a function called pickup_venom
  let pickup [false]                            ; this creates a local variable called pickup and sets it to false
  ask venom in-radius rad [                     ; this sets up a radius around the butterfly to the value of the global variable rad which we are
    set pickup true                             ; this sets the local variable pickup to true
    die                                         ; this removes the vpod from the map
  ]
  if pickup = true [                            ; if pickup is true then...
    set have_venom have_venom + 1               ; add 1 to the have_venom count on the butterfly
  ]
end
```

Once you have done this you need to call the function in the make_butterflies_move function as follows. Place this in a logical location within the function.

```
      pickup_venom ;++++++++++++++++++++++++++++++++++++++++++++; this calls the pickup_venom function
```

Step 19.3 – making the vpods have impact

To make the vpods have impact you need to add some lines of code to the people_function. Essentially you need to add a local variable to store the ID of the person hit, log the ID within the in-radius part of the code and add an if statement to use the vpod, kill the person and update the have_venom counter. The people function should now look as follows:

```
to-report people_function                       ; this creates a reporting function called people_function
  let seen [false]                              ; this creates a local variable called seen
  let hit [false]                               ; this creates a local variable called hit
  let person_hit 0    ;++++++++++++++++++++++++; this creates a local variable calles person_hit and sets it to 0

  ask peoples in-cone per_vis_rad per_vis_ang [ ; this sets up a vison cone on the butterfly with the parameters from per_vis_rad per_vis_ang and
    set color green                             ; this sets the color of the person detected within the vision code of the butterfly to green
    set seen true                               ; this sets the local variable called seen to true indicating that a person has been seen
  ]

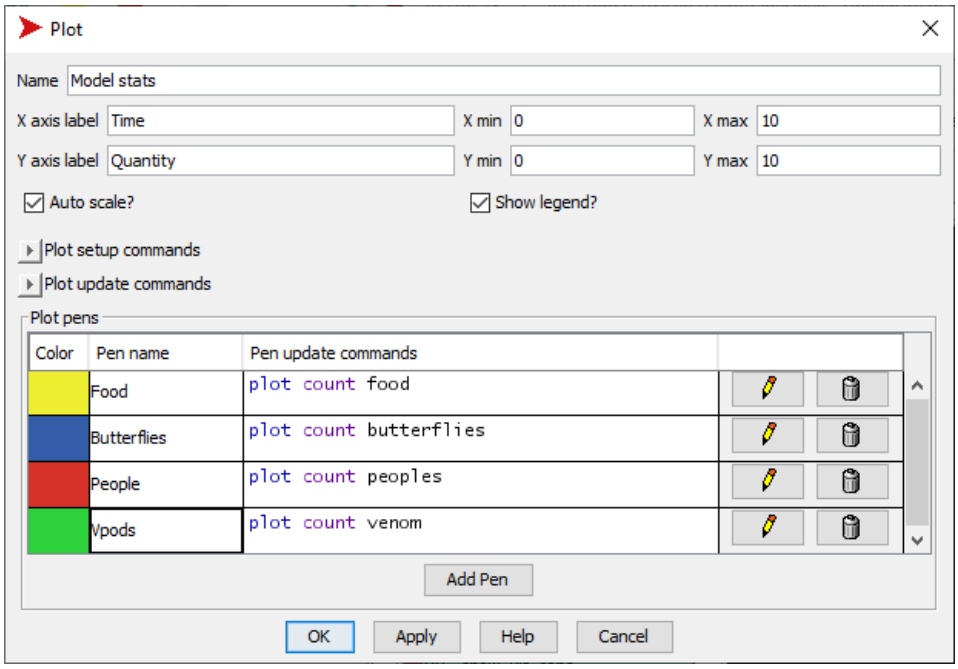
  ask peoples in-radius rad [                   ; this sets up a radius around the butterfly to the value of the global variable rad which we are
    set hit true                                ; this sets the local variable called hit to true indicating that a person has collided with the b
    set person_hit who ;++++++++++++++++++++++; this sets the local variable called person_hit to the individual who
    show person_hit
  ]

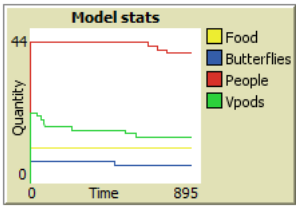
  ifelse seen = true [                          ; if then else statement based on the local variable seen, if seen = true then...
    set people_seen people_seen + 1             ; add 1 to the people_seen count
    set color white                             ; set color of butterfly to white
    ;right 180     ;-----------------------------; set heading of the butterfly to 180 (turn around to avoid!)
  ][                                            ; if seen = false...
    ;right (random bwr - (bwr / 2)) ;-----------; this turns the butterfly right relative to its current heading by a random degree number using
  ]

  if hit = true [                               ; if statement based on the local variable hit, if seen = true then...
    ifelse have_venom > 0 [ ;++++++++++++++++++; if have_venom is greater that 0 then...
      ask people person_hit [die] ;++++++++++++; kill off the person hit
      set have_venom have_venom - 1 ;++++++++++; remove 1 from the have_venom of the butterfly
    ][
      set people_hit people_hit + 1             ; add 1 to the people_hit count
      set color green                           ; set color of butterfly to green
      set health health - robustness ;++++++++++; adjust health of butterfly to health - collision penalty (robustness)
      adjust_vision_cone ;++++++++++++++++++++++; calls adjust_vision_cone to update the vision parameters based on heath changes
    ]
  ]
  report seen ;++++++++++++++++++++++++++++++++; return true or false based in local variable seen
end
```

Step 20 – using graphs to keep tabs on our model

As an easy visual indicator you can add a plot to your interface, this can be done by entering the details as per the screenshot below.





Step 21 – fixing an error in our existing code

The adjust_vision_cone code has a small error can you spot it? To fix this error we will create a butterfly variable called vis_rand and sent this to random 20 in the setup function. The code should look as follows.

```
butterflies-own [ people_seen people_hit    ; this creates 2 variables which will be used to count the total people seen and total people hit by each
health robustness speed_variation           ; this creates 3 variables for health, durability and speed
per_vis_rad per_vis_ang                     ; this creates variables for personalised vision cones
food_around_me closest_food                 ; this creates 2 variables to save the locations of food
have_venom                                  ; this creates a variable to store the amount of venom held
vis_rand                                    ; this creates a variable to store a stable vision cone random value
]
```

In the setup function add 1 line to the bottom of the create-butterflies code.

```
  create-butterflies number_of_butterfiles [; this creates the number of butterflies that your global variable states determined by the slider
    setxy random-xcor random-ycor           ; this sets the starting position of the butterflies to a random location in the world
    set color blue                          ; this sets the color of the butterflies to blue
    set size 10                             ; this sets the size of the butterflies to 10
    set shape "butterfly"                   ; this sets the shape of the butterflies to a butterfly

    ; making our butterflies unique
    set health 50 + random 50               ; this sets the health of the butterfly by adding 50 + a random allocation up to 50
    adjust_vision_cone                      ; this calls the adjust_vision_cone fuction to setup the vision cone
    set robustness random 10                ; this sets the robustness variable to a random value up to 10. lower means the butterfly is less affected
    set speed_variation random 10           ; this sets the speed_variation variable to a random value up to 10. the higher the value the faster the bu
    ;set heading 0                           ; this sets the starting heading of the butterfly to 0 (for demonstration of speed difference)
    pen-down                                ; this puts the pen down so you can see where the butterfly moves
    set vis_rand random 20
  ]
```

Now in the adjust_vison_cone function edit the code by replacing random 20 with vis_rand as seen below.

```
to adjust_vision_cone                                ; this creates a function called adjust_vision_cone
  if ((vis_rad + vis_rand)*(health * 0.01)) > 0 [    ; if the calculation if greater than 0 then...
    set per_vis_rad ((vis_rad + vis_rand)*(health * 0.01))  ; set the personal vision radius to factor in some randomness and health (less health =
  ]
  if ((vis_ang + vis_rand)*(health * 0.01)) > 0 [    ; if the calculation if greater than 0 then...
    set per_vis_ang ((vis_ang + vis_rand)*(health * 0.01))  ; set the personal vision angle to factor in some randomness and health (less health =
  ]
end
```

Changing this code means that the initialised random value stays the same rather than being adjusted every time the function is called.

Step 22 – creating more global variable to make our model more flexible

In order make our model easy to modify we will create some additional global variables to regulate day and night along with the timer reset. Add the following global variables.

```
patches-own [ solid ]                    ; this creates a variable for the patches

globals [rad                             ; this creates a global variable called r
daytime starting_color current_color     ; this creates 3 global variables relatin
color_adjust color_range                 ; this creates 2 global variables relatin
timer_reset ]                            ; this creates a global variable called f
```

Step 22.1 – setting our variables up

Setup the variable in the setup function to the following parameters.

```
  set timer_reset 1000                              ; this sets the global variable reset_timer to 1000
  set daytime true                                  ; this sets the global variable daytime to true
  set starting_color 85                             ; this sets the global variable starting_color to 85 which is blue
  set current_color starting_color                  ; this sets the global variable current_color to starting_color
  set color_range 5                                 ; this sets the global variable color_range to 5. the reason can be seen by looking at
  set color_adjust ( color_range / ( timer_reset + 10 )) ; this sets the global variable color_adjust to a range based on the variables above
```

Step 23 – adjusting the timer and adding functionality

The grow_more_food function currently resets the ticks in our model every 1000 ticks. Go to this function and adjust the code as follows. This will create the beginning of a system we can use to create day and night within our model.

```
to grow_more_food                        ; this creates a function called grow_more_food
  if ticks > timer_reset [ ;++++++++++++++  ; if the current number of ticks is greater than 100 then...
    ask patch random-xcor random-ycor [  ; ask a (1) patch in a random location (x, y coordinate) to do the following...
      sprout-food 1 [grow_food]          ; sprout (create new) food (1 in this instance) then call the grow_food function to set the parameters of
    ]
    ifelse daytime = true [              ; if global variable daytime is true...
      set daytime false                  ; set global variable daytime to false
    ][                                   ; otherwise...
      set daytime true                   ; set global variable daytime to true
    ]
    reset-ticks                          ; this resets the ticks counter back to 0
  ]
end
```

Step 24 – creating day and night in our world

To make our world more interesting we will revisit the reset_patch_color code. Edit the code as seen below to create night and day in the model.

```
to reset_patch_colour                          ; this creates a function called reset_patch_color
  ifelse daytime = true [                      ; if global variable daytime is true...
    set current_color current_color - color_adjust  ; adjust global variable current_color using color_adjust variable
  ][                                           ; otherwise...
    set current_color current_color + color_adjust  ; adjust global variable current_color using color_adjust variable
  ]
  ask patches [                                ; this asks all of the patches in the population to do what is in the brackets
    set pcolor current_color                   ; set color of all patches to global variable current_color
  ]
end
```

* Now test your model to see what happens. If all is working correctly you will have a background that changes color.

Step 25 – making day and night have an impact on the butterflies vision

In order for the day and night to have an impact on our butterflies range of vison we need to go back to the adjust_vison_cone code and modify this as follows.

```
to adjust_vision_cone                                                          ; this creates a function called adjust_visio
  if ((((vis_rad + vis_rand)*(health * 0.01))) - ((starting_color - current_color) * 2) > 0) [  ; if the calculation if greater than 0 then...
    set per_vis_rad (((vis_rad + vis_rand)*(health * 0.01))) - ((starting_color - current_color) * 2)  ; set the personal vision radius to factor in
  ]
  if ((vis_ang + vis_rand)*(health * 0.01)) > 0 [          ; if the calculation if greater than 0 then...
    set per_vis_ang ((vis_ang + vis_rand)*(health * 0.01))  ; set the personal vision angle to factor in some randomness and health (less health = l
  ]
end
```

After you have done this you need to call the adjust_vison_cone function in the make_butterflies_move function as this will continually change now.

```
    adjust_vision_cone                                  ; this calls the adjust_vision_cone function
    forward butterflies_speed + ( speed_variation * 0.1 )    ; moves butterfly forward by the butterflies_speed variable
```

It would also be good to remove the calls to the adjust_vison_cone function else where in the code as it will no longer be needed in the other locations. This can be done by searching using CTRL F.

Step 26 – creating buildings in the environment

In order to create solid structures in the environment we need to add a patch variable at the beginning of the code as follows.

```
patches-own [ solid ]                    ; this creates a variable for the patches to establish if it should be percived as solid
```

Step 26.1 – creating a function to setup our buildings

To create our building(s) we will create a function that we will then call in the setup. In this function we will change the color of the patches and also set the variable for each patch.

```
to draw_building                         ; this creates a function called draw_building
  ask patches [                          ; this selects all of the patches to follow a command
    set solid false                      ; this sets the patch variable solid to false for all patches
  ]
  ask patches with [ pxcor >= -30 and pxcor <= 30 and pycor >= -30 and pycor <= 30][ ; this selects only patches that meet the parameters
    set pcolor brown                     ; this sets the color of all of the patches selects to brown
    set solid true                       ; this sets the variable solid to true for all of the patches selected
  ]
end
```

Then call the function in setup.

```
  draw_building
```

Finally edit the reset_patch_colour function to only change the color of patches that are not solid.

```
to reset_patch_colour                          ; this creates a function called reset_patch_color
  ifelse daytime = true [                      ; if global variable daytime is true...
    set current_color current_color - color_adjust  ; adjust global variable current_color using color_adjust variable
  ][                                           ; otherwise...
    set current_color current_color + color_adjust  ; adjust global variable current_color using color_adjust variable
  ]
  ask patches [                                ; this asks all of the patches in the population to do what is in
    if solid = false [
      set pcolor current_color                   ; set color of all patches to global variable current_color
    ]
  ]
end
```

* Now test your model to see what happens. If all is working correctly you will have a brown square in the middle of your model, but see what happened when you run the model. Your agents will change the color of the brown square and go through it.

Step 26.2 – fixing the color changing issue

To fix this issue you need to revisit the show_visualisations function and add 2 if statements to check if the patch is solid or not as shown below.

```
to show_visualisations                          ; this creates a function called show_visualisations
  if show_col_rad = true [                      ; this will switch on the visualisation of the collision radius if the switch is set to true
    ask patches in-radius rad [                 ; this sets up a radius around the butterfly to the value of the global variable rad which we are
      if solid = false [ ;++++++++++++++++++++++; this checks the patch is not solid
        set pcolor orange                       ; this sets the patch color to orange
      ] ;++++++++++++++++++++++++++++++++++++++; closing if statment
    ]
  ]
  if show_vis_cone = true [                     ; this will switch on the visualisation of the vision cone if the switch is set to true
    ask patches in-cone per_vis_rad per_vis_ang [ ; this sets up a vision cone in front of the butterfly to the value of the global variables per_vi
      if solid = false [ ;++++++++++++++++++++++; this checks the patch is not solid
        set pcolor red                          ; this sets the patch color to red
      ] ;++++++++++++++++++++++++++++++++++++++; closing if statment
    ]
  ]
end
```

* Now test your model to see what happens. If all is working correctly the color changing issue is resolved but the agents in the model still pass through the solid object so this needs to be fixed.

Step 26.3 – making the object solid to our agents

We next need to create a function for our agents to see/hit the wall, we will call this detected_wall, the code is as follows.

```
to detect_wall                              ; this creates a function called hit_wall
  if [solid] of patch-ahead 1 = true [      ; if patch varible of 1 patch ahead is true then...
    right 180                               ; turn around to opposite direction
  ]
end
```

Once you have created this function call the function in both the make_people_move code and the make_butterflies_move code just before the piece of code that moves the agent forward.

```
    detect_wall ;++++++++++++++++++++++++++; this calls the detect_wall function
    forward people_speed                    ; this sets the speed at which the people move
```

And…

```
      detect_wall ;++++++++++++++++++++++++++++++++++++++++++++++; this calls the detect_wall function
      forward butterflies_speed + ( speed_variation * 0.1 )    ; moves butterfly forward by the butterflies_speed variable
```

* Now test your model to see what happens. If all is working correctly the agents will no longer pass through the building.